

Kompilator powinien to wyłapać

... ale jak?

Tobiasz Fic

Ericsson

22 Kwietnia 2026

Co to znaczy *lepiej*?

Ten sam efekt, mniej wykonanej pracy
(tzn. zużytych zasobów: pamięci/czasu)

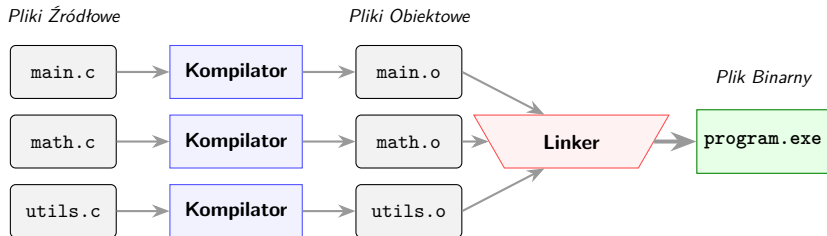
Aby działać lepiej, rób mniej

J. Bentley:

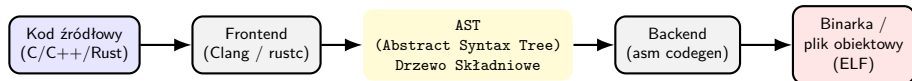
Czy mógłbym robić to...

- ❶ ... rzadziej?
- ❷ ... wcześniej?
- ❸ ... raz? (i wykorzystać wynik wiele razy)
- ❹ ... nigdy?

Toolchain



Anatomia Kompilatora

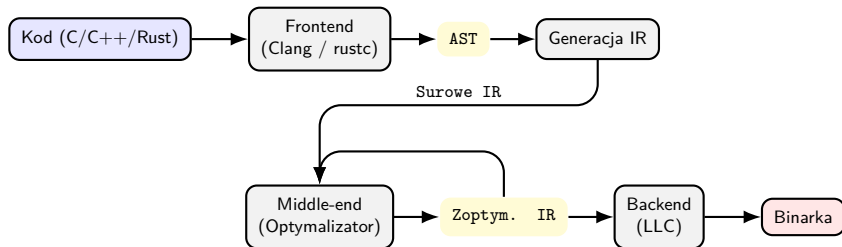


Język czytelny dla ludzi nie jest czytelny dla maszyn i vice versa

Język Pośredni (Intermediate Representation - IR)

AST z parsera jest zbyt złożone (zbyt blisko C), a kod assembly ma za dużo "niepotrzebnych" informacji sprzętowych (zbyt blisko krzemu).

Rozwiązanie: Język pośredni (np. LLVM IR, GIMPLE w GCC).



(stopniowe ulepszenia piszemy raz, działają dla każdej architektury i języka)

Format SSA (Static Single Assignment)

Każda zmienna może być przypisana *tylko raz*. Zakładamy że mamy nieskończoną ilość rejestrów.

Kod C:

```
x = 1;  
x = x + 2;  
y = x;
```

Format SSA (IR):

```
x_1 = 1;  
x_2 = x_1 + 2;  
y_1 = x_2;
```

Trywializuje to przepływ danych, umożliwia stosowanie algorytmów grafowych.

Spis Treści

Optymalizacji dokonuje się poprzez szereg wielu małych, stopniowych "ulepszeń"

1 Wstęp

2 Optymalizacje

- Pojedynczych wartości
- Logiki
- Pętli
- Funkcji
- Zależne od architektury
- Wynikające z Undefined Behavior (standardu C)

3 Jak pisać z myślą o kompilatorze?

Przed rozmową o optymalizacjach

- Odpowiedzią na większość bieżących pytań będzie "to zależy" (od architektury, kompilatora, standardu, heurystyki, reszty kodu etc.)
- Celem przekazu jest pokazanie co kompilator *może* robić
- Sprawdźcie sami: godbolt.org (Compiler Explorer)

Optymalizacje Pojedynczych Wartości

Zwijanie stałych (Constant Folding)

przed:

```
float perimeter(float r)
{
    const float PI = 3.14;
    return 2.0 * PI * r;
}
```

po:

```
float perimeter(float r)
{
    return 6.28 * r;
}
```

Kompilator wylicza wyrażenia złożone wyłącznie ze stałych już na etapie kompilacji, tak aby w binarce zastąpić je gotową wartością.

Tożsamości algebraiczne (Peephole)

Kompilator przegląda małe fragmenty kodu (okienko) i upraszcza matematykę:

Typ optymalizacji	Oryginał	Po optymalizacji
Tożsamość	$x \times 1 + 0 \longrightarrow x$	
Anihilator	$x \times 0 \longrightarrow 0$	
Redukcja siły	$x \times 8 \longrightarrow x \ll 3$	
Idempotencja bitowa	$x \& x \longrightarrow x$	
Prawa de Morgana	$!a \parallel !b \longrightarrow !(a \&\& b)$	

Optymalizacje Logiki

CSE: Eliminacja Wspólnych Podwyrażeń

Przed:

```
a = b * c + g();  
d = b * c * e;
```

Po:

```
tmp = b * c;  
a = tmp + g();  
d = tmp * e;
```

Działa również dla wywołań funkcji... pod warunkiem, że kompilator potrafi udowodnić, że funkcja `g()` **nie ma efektów ubocznych** (czyli dla tych samych argumentów zawsze zwraca to samo - np.

`__attribute__((const))`).

Short-circuiting

Standard C: W wyrażeniach `A && B` (oraz `A || B`), ewaluacja jest przerywana, gdy wynik całego wyrażenia jest już znany.

Klasyczny wzorec:

```
if (ptr != NULL && ptr->
    status == READY) {
    // ...
}
```

Tłumaczy się na:

```
if (ptr != NULL) {
    if (ptr->status ==
        READY) {
        // ...
    }
}
```

Dzięki temu odczyt `ptr->status` jest bezpieczny - unikamy segfault.
ALE...

Short-circuiting (cont.)

Maszyna nie lubi niespodzianek!

```
if (a < b && b < c) {  
    // ...  
}
```

```
bool test_1 = a < b;  
bool test_2 = b < c;  
bool test = test_1 &&  
           test_2  
if (test) {  
    // ...  
}
```

Robi wszystko co może aby uniknąć niepotrzebnych skoków.

Scalanie warunków

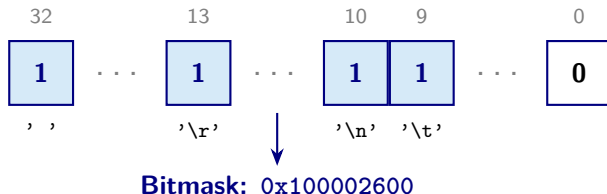
```
bool is_whitespace(char c) {  
    return c == '\r' || c == '\n' || c == ' ' || c  
           == '\t';  
}
```

Mamy tu potencjalnie 4 kosztowne instrukcje skoku (branch)

Jak to usprawnić?

Scalanie warunków (cont.)

Zamiast 4 ścieżek kompilator traktuje `c` jako indeks i wykona tylko 1 operację.



```
bool is_whitespace(char c) {  
    // Ideally at compile time  
    static const uint64_t lookup = (1 << '\r')  
        | (1 << ' ') | (1 << '\n') | (1 << '\t');  
    return (lookup >> c) & 1; // 0(1)  
}
```

Kolejność warunków a Branch Prediction

Maszyna nie lubi niespodzianek

Branch Predictor stara się zgadnąć, czy wejdziemy w `if`, czy w `else`.

```
if (is_error) { [[unlikely]]  
    // Rzadki przypadek (1%)  
    handle_error();  
} else { [[likely]]  
    // Główna logika (99%)  
    process_data();  
}
```

Narzędzia do zasugerowania prawdopodobieństwa:

- Standard **C23** / **C++20**: `[[likely]]` oraz `[[unlikely]]`.
- GCC/Clang: `__builtin_expect(expr, value)`.

Optymalizacje Pętli

Hoisting (Loop Invariant Code Motion)

Kompilator wyciąga niezmiennie obliczenia przed pętlę.

Przed:

```
for(int i=0; i<n; i++)  
    out[i]=in[i]+a*b;
```

Po:

```
x = a*b;  
for(int i=0; i<n; i++)  
    out[i]=in[i]+x;
```

Ale... co jeśli a i b to wskaźniki/referencje?

Problem Hoistingu: Aliasing wskaźników

Kompilator zakłada że...

dwa wskaźniki mogą pokazywać na ten sam adres!

```
void process(int *out, int *in,
             int *factor, int n) {
    for(int i=0; i<n; i++) {
        // Czy możemy wyciągnąć to przed pętlę?
        int x = *factor * 2;
        out[i] = in[i] + x;
    }
}
```

Nie! Jeśli `out == factor`, to zapis do `out[i]` zmienia wartość `*factor` w kolejnej iteracji! Kompilator odczyta z (i zapisze do) pamięci za każdym razem

Rozwijanie Pętli (Loop Unrolling)

Przed:

```
for(int i=0; i<4; i++) {  
    buf[i] = 0xe;  
}
```

Po:

```
buf[0] = 0xe;  
buf[1] = 0xe;  
buf[2] = 0xe;  
buf[3] = 0xe;
```

Plus: Uniknięcie kosztu inkrementacji licznika i skoków warunkowych.

Clang/NVCC: `#pragma unroll`

Haczyk: Zwiększa rozmiar kodu (kłóci się z flagą `-Os`).

Kompilator wie najlepiej, czy warto (np. rozwinie tylko małe pętle lub takie o stałej liczbie iteracji).

Fuzja pętli (Loop Fusion)

Przed

```
for(int i=0; i<N; i++)  
{  
    do_thing1(i);  
}  
  
for(int i=0; i<N; i++)  
{  
    do_thing2(i);  
}
```

Po:

```
for(int i=0; i<N; i++)  
{  
    do_thing1(i);  
    do_thing2(i);  
}
```

Plus: Połowa inkrementacji ($i++$), porównań warunku wyjścia ($i<N$).

Plus: Połowa skoków (branches) na koniec pętli, co odciąża branch predictor i oszczędza czyste cykle procesora.

Haczyk: jeżeli `do_thing1` i `do_thing2` są na tyle duże że nie mieszczą się w cache'u kompilator może zdecydować się na odwrotność tej optymalizacji :))

Wektoryzacja (Auto-Vectorization / SIMD)

Kompilator przekształca kod skalarny na instrukcje wektorowe (**S**ingle **I**nstruction **M**ultiple **D**ata, np. ARM NEON).

Przed:

```
for(int i=0; i<N; i++)  
    c[i] = a[i] + b[i];
```

Po:

```
vld1q_s32(...) // ld 4a  
vld1q_s32(...) // ld 4b  
vaddq_s32(...) // add vec  
vst1q_s32(...) // save 4c
```

Plus: pisanie SIMD jest zazwyczaj mocno zależne od architektury, tu kompilator robi to za nas

Haczyk: kompilator nie ma gwarancji że tablice a, b i c na siebie nie nachodzą (restrict!).

Haczyk 2: co jeżeli $N \% 4 \neq 0$?

Dla ciekawych: <https://arxiv.org/abs/1902.08318>

Optymalizacje Funkcji

Inlining

to wstawienie funkcji w miejsce wywołania (eliminacja operacji na stosie i skoków).

Ale co ważniejsze: *Inlining umożliwia inne optymalizacje!*

przed:

```
int obj_size(int scale)
{
    return 100 * scale;
}

// ...
int x = obj_size(0);
```

po:

```
// Funkcja wklejona:
int x = 100 * 0;

// Umożliwia zwijanie stałych:
int x = 0;
```

Nagle argument funkcji (scale) stał się stałą w tym konkretnym wywołaniu. Kompilator optymalizuje całą gałąź do jednego zera!

Wywołania ogonowe (Tail Call Optimization)

Jeśli ostatnią operacją w funkcji jest zwrócenie wyniku innej funkcji, kompilator nie tworzy nowej ramki stosu (Stack Frame).

Przed:

```
int f_wrapper(int a) {  
    // Zrób coś...  
    a = a + 1;  
  
    // Ostatnia operacja!  
    return f_worker(a);  
}
```

Przed TCO (asm):

```
CALL f_worker  
RET
```

Po

```
JMP f_worker
```

Zmniejsza ryzyko przepełnienia stosu, zwłaszcza przy maszynach stanów i rekurencji.

Optymalizacje zależne od architektury

Rozpoznawanie idiomów

Kompilator na etapie IR stara się odgadnąć Twoje intencje, a nie tylko ślepo tłumaczyć C na ASM. Zna trochę powszechnych wzorców.

Przykład: Liczenie zapalonych bitów (Popcount)

```
int count = 0;
while (n) {
    count += n & 1;
    n >>= 1;
}
```

Na architekturze x86 wygeneruje jedną instrukcję: POPCNT.

Od C23 i C++20 mamy natywne abstrakcje (np. `std::popcount` z `<bitops>`), ale kompilator łapie to sam w starszym kodzie!

Fuzja instrukcji sprzętowych: ARM

Kompilator doskonale zna sprzęt docelowy i jego specjalistyczne koprocesory (np. jednostki do obliczeń adresów lub DSP).

Przykład 1 (ARM DSP / Cortex-M4): Multiply-Accumulate

```
a += b * c;
```

Zamiast osobnego mnożenia i dodawania (2 cykle), generuje instrukcję MLA (Multiply Load Accumulate) - wykonaną w **1 cyklu zegara**.

Od C++11: `std::fma` wymusza to zachowanie

Fuzja instrukcji sprzętowych: x86

Obliczenia na wskaźnikach (Load Effective Address)

```
int x = a + b * 4 + 8;
```

↓

```
lea rax, [rdi + rsi * 4 + 8]
```

Działania na liczbach całkowitych delegowane są do jednostki *w teorii* przeznaczonej do adresowania pamięci. W przypadku mnożenia:

```
int x = a * 5;
```

↓

```
lea rax, [rdi + rdi * 4]
```


Optymalizacje wynikające z Undefined Behavior (UB)

Kompilator a UB

Kompilator zakłada że...

W kodzie nie ma Undefined Behavior (UB)

(tj. zdefiniowany przez standard języka, w naszym przypadku C/C++).

Jeśli warunek wymagałby wystąpienia UB, kompilator zakłada, że będzie to zawsze fałsz.

UB: signed overflow

Wykrywanie przepełnienia

```
int inc_safe(int a) {  
    if (a + 1 < a) {  
        return ERROR;  
    }  
    return a + 1;  
}
```

Logika kompilatora:

Przepełnienie na typie `signed int` to w C/C++ UB. Zgodnie ze standardem, to się po prostu *nie może* wydarzyć.

Zatem matematycznie $a + 1$ **zawsze** jest większe od a .

Kompilator wycina całego if'a

UB: "Podróż w czasie"

Spóźniony Null-Check

```
int process(Data* ptr) {  
    int val = ptr->value;  
  
    if (ptr == NULL) {  
        return ERROR;  
    }  
  
    return val * 2;  
}
```

Logika kompilatora:

W linii 1 nastąpiła dereferencja wskaźnika. Gdyby ptr było NULL-em, wystąpiłoby UB. Kompilator zakłada, że UB nie wystąpi, wyciąga wniosek: ptr **na pewno nie było NULL-em**.

Kompilator wycina całego if'a

Pisanie z myślą o kompilatorze

Przejrzysty kod to szybki kod

- **Intencja ponad triki:** Kompilatory są optymalizowane pod kątem idiomatycznego kodu. Koszty abstrakcji ułatwiającym programistom życie powinny być małe.
- **Efekty uboczne:** Im mniej funkcji zmienia stan globalny, tym więcej zostawiamy kompilatorowi miejsca do pracy.

"Zazwyczaj kod, który łatwo zrozumie człowiek, łatwiej analizuje też maszyna."

const to kontrakt, nie dekoracja

Obietnica niezmienności: Dzięki `const` kompilator wie, że wartość jest niezmienna.

Jeżeli nie damy mu tej gwarancji, musi udowodnić że wartość ta nie ulega zmianie aby ten fakt wykorzystać.

Aliasing i słowo restrict

Kompilator jest paranoikiem. Jeśli widzi:

```
void add(int *a, int *b, int *res) {  
    // ...  
}
```

musi założyć, że wskaźniki mogą wskazywać na to samo miejsce w pamięci.

Uniemożliwia/utrudnia to szereg optymalizacji.

Pamiętać o słówku restrict

Daj kompilatorowi miejsce do pracy

- **Ręczny inline:** Dzisiejsze kompilatory zazwyczaj lepiej zarządzają budżetem pamięci niż człowiek.
- **Odwijanie pętli:** Ręczne odwijanie psuje heurystyki prefetchera procesora i pogarsza czytelność
- **Bit-hacks:** Używanie $x \ll 1$ zamiast $x * 2$ nie niesie za sobą benefitów. Kompilator dobierze najlepszą instrukcję pod konkretny procesor (np. LEA).

Narzędzia

- **Compiler Explorer (godbolt.org)**

Pozwala na żywo podglądać kod w C/C++/Rust i zestawiać go z wygenerowanym Assembly dla dziesiątek kompilatorów i architektur.

- **Zrzucanie Języka Pośredniego (IR)**

- ▶ **Clang:** `clang -O2 -emit-llvm -S main.c` (generuje .ll)
- ▶ **GCC:** `gcc -O2 -fdump-tree-optimized main.c`

- **perf-record/perf-report/google-benchmark**

krótsze assembly nie zawsze znaczy szybsze

- **Analiza Statyczna**

`clang-tidy`, `cppcheck` czy `clippy` (Rust)

• Książki

- ▶ *"Compilers: principles, techniques, and tools"* – Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman (frontend, analiza semantyki)
- ▶ *"Engineering a Compiler"* – K. Cooper, L. Torczon (backend)

• Artykuły

- ▶ *"What Every C Programmer Should Know About Undefined Behavior"* – Chris Lattner (twórca LLVM)
- ▶ *"Future Directions for Optimizing Compilers"* – Nuno P. Lopes, John Regehr
- ▶ *"Bit Twiddling Hacks"* –
<https://graphics.stanford.edu/~seander/bithacks.html>

Dziękuję :))